

Lecture 1 – The Math, Gently

Why the value is finite, where the Bellman equation comes from, and why our methods converge — in plain terms.

A short companion to the lecture. We keep only the few facts worth knowing *why*, and explain each in words first, with at most a line or two of algebra. Nothing here is needed to *use* the methods — it is here so the ideas feel inevitable rather than magical.

$$V(s) = r + \gamma V(s')$$

1 Why the value is a finite number

The return adds up infinitely many rewards: $G = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$. Could it blow up to infinity? No — as long as $\gamma < 1$.

Each step counts a little less than the one before (by the factor γ). Adding up a shrinking sequence gives a finite total, just like $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 1$. So “value” is always a real, comparable number.

If every reward is at most $R_{\max}$, then $|G| \leq R_{\max}(1 + \gamma + \gamma^2 + \dots) = \frac{R_{\max}}{1 - \gamma}$. That is the whole reason $\gamma < 1$ matters.

2 Where the Bellman equation comes from

Split the return into “the first reward” and “everything after”:

$$G_t = r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma G_{t+1}.$$

Take the average (expectation) of both sides and you get the Bellman equation:

$$V(s) = \mathbb{E}[r + \gamma V(s')].$$

“The value of where I am is the reward I just got, plus the discounted value of where I land next.” Everything in the course is a way of solving, sampling, or approximating this one line.

3 Why value iteration converges

Value iteration just applies the Bellman backup over and over. Why is it guaranteed to settle on the right answer?

The backup is like a photocopier set to 90%. Feed in *any* two guesses for the value function, copy them through one backup, and they come out closer together — by at least the factor γ . Repeat, and every guess is squeezed toward the *same* point. That point is the true value V^* .

Concretely, if your current error is E , after one sweep it is at most γE , after k sweeps at most $\gamma^k E$ — which goes to 0. (The formal name is a “contraction”; the one-line statement $\|\mathcal{T}U - \mathcal{T}V\|_\infty \leq \gamma \|U - V\|_\infty$ is all it means.)

This is why a far-sighted agent (γ near 1) learns *slowly*: γ^k shrinks gently when $\gamma \approx 1$. You will feel this when tuning γ for DQN.

4 Monte Carlo and TD, side by side

Both estimate the *same* V ; they differ only in what they use as the “target” to move toward.

- **Monte Carlo** averages *real, observed* returns. Update toward G_t : $V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$. Unbiased (the average of real returns is correct), but noisy, and you must wait for the episode to end.
- **Temporal difference** doesn't wait — it *bootstraps*, replacing the unknown tail with its current guess of the next value. Update toward $r + \gamma V(s')$: $V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$. Lower variance (one step of real randomness, not a whole episode), at the cost of a little bias early on.

The bracket $r + \gamma V(s') - V(s)$ is the **TD error** — “how surprised was I?”. Next lecture it becomes, almost unchanged, the loss we minimise in DQN. The “incremental average” shape $new \leftarrow old + \alpha (target - old)$ is the template for every update in this course.

Appendix – Answers to the “Questions to think about”

The end-of-section reflective questions from the slides, with model answers (a key for discussion, not the only correct phrasings).

Section 2

1. **We claimed DQN, AlphaGo, PPO and GRPO are “the same recursion on a different surface.” Pick any two and state precisely what the state, action, and reward are in each.**

DQN: state = stack of Atari frames, action = joystick move, reward = change in game score. AlphaGo: state = board position, action = a legal move, reward = +1 win / -1 loss. PPO-for-RLHF: state = prompt + tokens so far, action = next token, reward = reward-model score. GRPO: same as PPO but the reward is group-relative. All four estimate value through the same $V(s) = r + \gamma V(s')$ recursion — only the meaning of s , a , r changes.

2. **Bellman had the equation in 1957, yet RL didn’t work at scale until ~2013. What changed in those decades? (Hint: it wasn’t the math.)**

Compute, data, and deep function approximation — not the math. GPUs, large amounts of (often simulated) experience, and ConvNets made it possible to represent value over enormous state spaces. Bellman’s equation was always correct; until ~2013 there was just no scalable way to store V over pixels.

3. **TD-Gammon (1992) already combined a neural net + self-play + TD learning. So why did it take ~20 more years to get DQN?**

TD-Gammon worked partly because backgammon’s dice smooth the value surface and make self-play explore naturally — properties that do not transfer to most games. DQN needed the stability engineering (experience replay, target networks, convolutional features) and the compute to make TD + a neural net work on high-dimensional, deterministic, sparse-reward games. The idea existed; the tricks and hardware did not.

Section 3

1. **A single Atari frame is not Markov. Why not? What’s the standard fix, and what does that fix teach you about the word “state” in general?**

A single frame shows position but not velocity — you cannot tell which way the ball is moving, so the future is not determined by the present frame alone. Standard fix: stack the last 4 frames. The lesson: ‘state’ is not ‘what you observe’, it is ‘whatever makes the future independent of the past’ — you engineer the state until the Markov property holds.

2. **Give a real task where the Markov property is badly violated. How would you enlarge the state so it holds again — and what does that cost you?**

Almost any partially-observed task: poker (you cannot see opponents’ cards) or a robot with a narrow camera. Enlarge the state by adding history or a belief over the hidden part. The cost: a much larger state space, more to learn, and you can never inject information that is genuinely unobservable.

3. **Describe, in one sentence each, the agent you get with $\gamma = 0$ versus $\gamma = 0.999$. When would a low γ actually be the right choice?**

$\gamma = 0$ gives a purely myopic agent that maximises only the immediate reward and ignores all consequences. $\gamma = 0.999$ gives a far-sighted agent that weights rewards hundreds of steps away almost equally. A low γ is right when the task truly is immediate (one-step decisions) or when long horizons add only noise and instability.

4. **The reward function is the only place you tell the agent what you want. Invent a reward for "drive safely" that an agent could obviously game. What does that reveal?**

Example: reward = +1 for every second without a crash. The agent games it by driving extremely slowly or never leaving the driveway — technically 'safe', useless. It shows that the reward specifies what you measure, not what you mean; any proxy can be exploited. This is reward hacking.

Section 4

1. **If you could only be handed one of $V(s)$ or $Q(s,a)$, which one lets you actually act without any extra computation — and why?**

2. **Two states have the same value under some policy π . Does that mean they're equally good under the optimal policy? Argue both directions.**

No. Equal value under one policy π says nothing about the optimal values: a different policy could exploit one state far more than the other. Conversely, two states can still tie under the optimal policy (e.g. both one step from the goal). So same-value-under- π neither implies nor forbids same-value-under-optimal.

3. **Why is value defined as an expectation over returns rather than the return of a single rollout? What goes wrong if you trust one trajectory?**

A single rollout is one sample from a random process (stochastic policy and/or environment); it can be very lucky or unlucky. Value is the mean over all such rollouts. Trust one trajectory and you fit noise — the estimate has huge variance and can be arbitrarily wrong.

4. **The relation $V^*(s) = \max_a Q^*(s,a)$ looks innocent. What does it quietly assume about how the agent behaves after the first step?**

Section 5

1. **The backup "looks to the future to value the present." Causally the future depends on now — so in what sense is the backup a causal inversion, and why is that legitimate?**

The backup writes $V(s)$ in terms of $V(s')$ — it values the present using values of the future. It is not predicting the future from the past; it defines a state's value by the values of states it can reach, then iterates until consistent. Legitimate because it is a fixed-point equation $V = TV$, not a causal claim — we solve for the self-consistent V .

2. **Value iteration is just the same backup applied over and over. Why is it guaranteed to converge? What property of the Bellman operator forces it? (We prove this in the companion notes.)**

Because the Bellman optimality operator is a γ -contraction in the sup-norm: each application moves any two value estimates strictly closer, by a factor γ . The Banach fixed-point theorem then guarantees a unique fixed point and geometric convergence. (Proved in the companion PDF.)

3. **A Go board has $\sim 10^{170}$ positions — you can never enumerate them. How does AlphaGo still apply the Bellman backup without a table over all states?**

It never enumerates states. AlphaGo approximates V (and the policy) with a neural network and applies the backup only along states actually visited by tree search and self-play, generalising to unseen boards. The backup is local — it needs only the current state and its

successors, not the whole space.

4. **Define "bootstrapping" in one sentence. Then explain why it is simultaneously the source of TD's speed and the source of its instability.**

Bootstrapping = updating an estimate using other current estimates instead of waiting for the true return. It is fast because you can update every step without finishing the episode; it is unstable because errors in the estimates feed back into the targets, and with function approximation those errors can amplify (the deadly triad).

Section 6

1. **You're handed a brand-new environment with no model of its dynamics. Of DP, MC, and TD — which are even available to you, and why is one ruled out?**

MC and TD are available; DP is ruled out because DP needs the model $P(s' | s, a)$ and R up front, which you do not have. MC and TD learn directly from sampled experience.

2. **MC and TD both converge to the same V^* . So why does TD typically get there in roughly 10× fewer episodes? What is it exploiting that MC throws away?**

TD bootstraps: it reuses its current estimate of the next state's value immediately, so a reward's information propagates after a single transition. MC discards that intermediate structure and waits for the whole-episode return, so each update is one noisy full-episode sample — far less information per step.

3. **Name one concrete situation where you'd actually prefer Monte Carlo over TD. (Hint: think about bias, and about episodes that are short and cheap.)**

Prefer MC when episodes are short and cheap and you want zero bootstrap bias — or early in training when the value estimates are still unreliable, so bootstrapping would inject bad bias. MC is unbiased; when variance is not the bottleneck, that can win.

4. **Next lecture we replace the value table with a neural network. Predict: which term in the TD update $r + \gamma V(s') - V(s)$ becomes the "loss" we minimise?**

The TD error itself: $r + \gamma V(s') - V(s)$. We turn the target $r + \gamma V(s')$ into a regression label and minimise the squared TD error — and that becomes the DQN loss.